Optimizing the OV7670 camera frame rate

Karthik Ganesan

Introduction

In this lab, you will be optimizing the Nucleo + OV7670 setup from Lab 5 to increase the frames per second (FPS) of the video stream sent from the Nucleo board to the computer. At the end of this lab document, you will find a list of items that you must show your TA when being marked during your in-lab demo.

1. Initial Set Up

Use your implementation from Lab 5 as a baseline. Similar to Lab 5, you will use the `serial_monitor` program to visualize the images. However, this is NOT the same program from before; this version has extra functionality that we will use in this lab. You must modify your code from Lab 5 to start all frame transmissions with `"\r\n!START!\r\n"` and end with `"!END!\r\n"`. This is because we will be sending a variable number of bytes per frame and the `serial_monitor` program needs to know when the frame is done.
First, download `serial_monitor_lab_06.zip`, extract it and make sure you have permissions to execute the `.exe` file inside. Open a command line interface and navigate to the folder where you extracted the `serial_monitor_lab_06.exe` file. Then, run `.\serial_monitor_lab_06.exe --help`. You should see the following:

```
> .\serial_monitor_lab_06.exe --help
Usage: serial_monitor_lab_06.exe [OPTIONS]

  Display images transferred through serial port. Press 'q' to close.

Options:
  -p, --port TEXT          Serial (COM) port of the target board
  -br, --baudrate INTEGER  Serial port baudrate
  --timeout INTEGER        Serial port timeout
  --rows INTEGER           Number of rows in the image
  --cols INTEGER           Number of columns in the image
  --preamble TEXT          Preamble string before the frame
  --delta_preamble TEXT    Preamble before a delta update during video
                           compression.
  --suffix TEXT            Suffix string after receiving the frame
  --short-input            If true, input is a stream of 4b values
  --rle                    Run-Length Encoding
  --help                   Show this message and exit.
```

Verify that you can run the new `serial_monitor` in a similar way to Lab 5. On your computer, run:

```
.\serial_monitor.exe -p <Nucleo board COM port>
```

Use Keil μVision to send through serial port:

- String `"\r\n!START!\r\n"`.
- The character `0x00` 25,056 times (144×174).
- String `"!END!\r\n"`.

Note the addition of the closing string `"!END!\r\n"`. This is needed because in this lab you will work with frames of varying byte sizes, and you need a specific byte sequence for `serial_monitor_lab_06` to identify the end of a transmission.
Run `serial_monitor_lab_06` for at least 2 minutes and fill in in Table 1 with the typical, minimum and maximum FPS reported by the tool end of this handout. You may ignore the first few frames until reaching a steady state.
If your implementation is correct, a graphical window will open showing a black rectangle. Next, modify your code to output character `0xFF` instead of `0x00`. The `serial_monitor` should show a completely white rectangle. You may experiment with shades of gray, or alternating patterns (e.g., 0x55 or 0xAA).
When you can reliably send and visualize artificially generated images, you may proceed to the next section.

2. Sending over UART using DMA

The biggest issue with the code you wrote in Lab 5 is that sending full image frames over the Serial Port is very time consuming. One reason for this is that `HAL_UART_Transmit()` is a blocking function; the CPU does not move past that function until all data has been transmitted. As a result, the UART hardware sits idle for all the cycles that the main execution thread is not running `HAL_UART_Transmit()`. We can improve this by using the DMA to transmit data over the UART with non-blocking functions.
In `config.c`, you will see that `print_msg` now uses the `HAL_UART_Transmit_DMA()` function instead. Note that the DMA transmit function can support much longer buffers than its blocking counterpart, and you can send an entire frame in a single function-call. While this makes it possible to send data over UART without direct CPU involvement, you now need a way to check if the UART is free before you call `print_msg`. You can check if the UART is free to begin a transmission by calling `HAL_UART_GetState(&huart3)`. If this function returns `HAL_UART_STATE_READY`, you can sent another frame over UART.
Once you had modified your code, measure the FPS you get with `serial_monitor` and enter your results in Table 1.

3. Compressing the data

Another way for us to increase FPS is to *compress* the data we are sending. Compression is a technique for reducing the size of data by removing redundant information. Images are highly amenable to compression as image-data contains significant amount of redundant information. There are two main types of compression: 1) lossless compression, where all the information is retained but we reduce data size by removing only redundant information and 2) lossy compression, where we remove some less-important data to reduce size. In this lab, you will implement one of each type of compression: *run-length encoding* (a lossless technique) and *data truncation* (a lossy technique).

3.1 Data truncation

We represent each pixel with an 8-bit value, which means that each pixel can have one of 256 possible values. To reduce the amount of data per frame, we can reduce this by only sending the 4-most significant bits from each pixel. This limits the resolution of our image (i.e., from $2^8 = 256$ to just $2^4 = 16$ values per pixel), but cuts the amount of data we have to send by 50%. Modify your code to 'pack' the 4-most significant bits from two pixels such that each byte sent over the UART contains the information for 2 pixels, shown in Fig. 1.

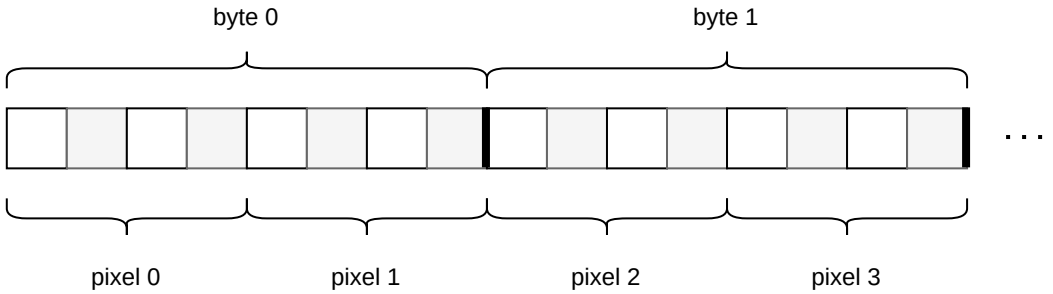

Figure 1. 4-bit pixels diagram.

Run the serial monitor with the flag `--short-input` to view an image streamed using this format. Measure the FPS and fill in your results in Table 1. If your implementation is correct, the image will be coarser, but frames per second will roughly double.

3.2 Run-Length Encoding

Run-Length Encoding (RLE) replaces repeated copies of the same value in the data as a single value and a count. For example, the string: `AAAAAAAABBACCCAA` is represented in RLE as `A8B2A1C3A2`. Note that RLE does not always result in a shorter string than the original, for example the string `ABC` is encoded as `A1B1C1` which is twice as long as the original.

RLE is well suited for certain types of images such as computer icons, where adjacent pixels are likely to share the same shade or colour. However, RLE does not perform well for continuous tone images such as photographs coming from the OV7670 camera. This is because even though adjacent pixels might seem to be the same color, they differ slightly in the actual 8-bit pixel value, which limits the efficacy of RLE. However, even for frames coming from the camera, the upper-bits of each pixel are more likely to be the same between adjacent pixels, compared to the lower-bits. Therefore, to improve the effectiveness of RLE, we can combine RLE with data truncation.

3.3 Combining techniques

Modify your code to implement RLE, and send the encoded buffer through serial port. Each byte transmitted through serial port will consist of a pixel value in the 4 most-significant bits, and the number of consecutive pixels with the same value in the 4 least-significant bits as shown on Fig. 2. As we only use 4 bits to keep count of the number of repeated pixels, a sequence of repeated pixels longer than 15 should be sent as a separate byte.

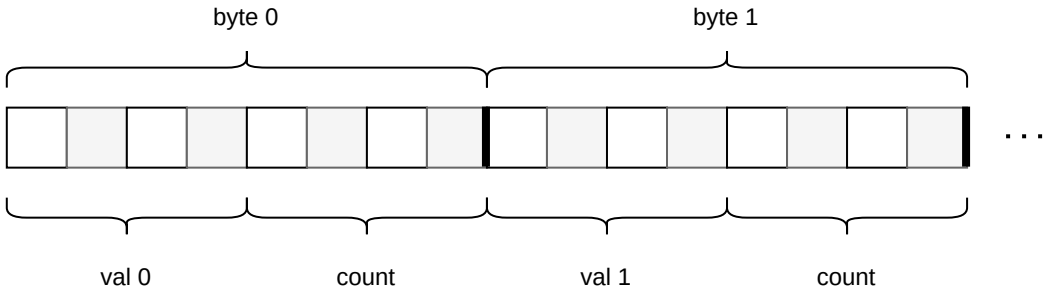

Figure 2. 4-bit pixels RLE diagram.

Run the serial monitor with the flags `--short-input --rle` to view an image stream this format. Measure FPS and fill in your results in Table 1. A correct implementation should be able to achieve FPS higher than 1.0 in the typical case.

4. (optional) Video Compression

So far you have focused on optimizing the transmission of individual frames, and you have taken advantage of the spatial correlation between pixels. Because the frames are part of a video, there is one remaining dimension we use for compression: time.

Pixels in consecutive frames of a video are typically highly correlated. In other words, we expect pixels in one place in one frame (i.e., frame A) to be the same or very similar to the same pixels in the next frame (i.e., frame B). We can take advantage of this to send a smaller transmission with the changes instead of full frames. If we calculate the per-pixel differences between frames A and B, we expect to get a lot of 0s. This makes this difference much more efficient to compress using RLE.

Implement a function to calculate the difference for every pixel between the current frame and the one before it. You must still use data truncation to only consider the 4 most significant bits for each pixel. Once you have this array of differences, compress it using RLE and send the result through the serial port. When sending just the differences, start the transmission with `"\r\n!DELTA!\r\n"` instead of `"\r\n!START!\r\n"` to tell the serial monitor that this transmission contains a partial update of a frame. You cannot keep sending just differences as doing this for too many frames will lead to a major drop in quality. Therefore, once every N frames, you should send a full frame. For $N = 5$, measure the FPS you achieve and fill in your results in Table 1.

Observe how the frame rate changes if you capture a moving object from a static image. Why do you think this happens? You should be ready to answer questions from your TA about this behaviour if asked.

Try it yourself: How many partial frames can you send before you notice that your video quality starts degrading? How much does the content of the video (i.e., static video vs. video with lots of movement) affect this number N? Can you think of a way to automatically calculate N based on the calculate differences?

5. In-lab Demo

During your lab session, you must show your TA the camera streaming a video with 4-bit pixels and run-length encoding. If you are unable to get this working, you can show your TA the camera streaming a video with just 4-bit pixels. Report your results in Table 1.

Table 1. Frames per second results.

	Min. FPS	Typ. FPS	Max. FPS
Baseline			
UART with DMA			
4-bit pixels			
4-bit pixels with RLE			

For full marks, your images should show little or no screen tearing.
Your TA may ask questions about your solution. Be prepared to explain your design decisions and your code.